

Practica final integradora

Juan Diego Cortes Galvis (clase de la mañana)

Leidy Dayhana Roldan Osorio (clase de la tarde)

Universidad EAFIT, Facultas De Ciencias Aplicadas E Ingeniería

Análisis y diseños de algoritmos

Carlos Alberto Alvarez Henao

8 de mayo de 2026

Repositorio de la práctica: https://github.com/cortes1443/ADA_PF_CORTES_ROLDAN

1. Introducción

Este proyecto implementa un pipeline de análisis y solución sobre el dataset de churn de Telco . El objetivo general es aplicar tres paradigmas de análisis de algoritmos sobre distintas partes: Divide y Vencerás para ordenar y buscar, Codicioso para construir un árbol de expansión mínima y Programación Dinámica para resolver una versión de mochila 0-1. La elección de cada paradigma responde adecuadamente al ordenar y buscar, en donde se requieren eficiencia sobre secuencias, el MST admite una solución óptima con una estrategia codiciosa globalmente válida, y la mochila 0-1 requiere explorar combinaciones con garantía de optimalidad, lo cual se logra con tabulación dinámica.

El dataset contiene información de clientes de una empresa de telecomunicaciones, incluyendo variables como tenure, MonthlyCharges, TotalCharges y churn. A partir de estos datos se construyen estructuras para cada módulo: un arreglo ordenado para búsquedas binarias, un grafo ponderado para Kruskal y un subconjunto filtrado de solicitudes para la mochila. Esto permite comparar, sobre un mismo conjunto de datos reales, cómo se comportan distintos paradigmas de algorítmico.

2. Descripción del dataset

El archivo CSV cargado contiene 7043 registros. Durante la carga se detectaron 11 valores nulos en TotalCharges, los cuales deben tratarse explícitamente porque esa columna se usa como atributo numérico dentro del procesamiento. En la implementación, esos valores se manejan durante el parsing para evitar que el flujo del programa se interrumpa o produzca resultados inválidos.

Las variables más relevantes para el proyecto son tenure, MonthlyCharges, TotalCharges y Churn. tenure representa el tiempo de permanencia del cliente en meses. MonthlyCharges representa el cargo mensual, mientras que TotalCharges representa el cargo acumulado. Churn indica si el cliente abandonó o no el servicio.

La construcción del grafo para el módulo de Kruskal se realiza agrupando las solicitudes en 20 nodos. Cada solicitud se asigna al grupo $i \bmod 20$, donde i es su posición en el arreglo cargado. Para cada grupo se calcula el promedio de MonthlyCharges, y luego se crea una arista entre cada par de nodos distintos. El peso de la arista se define como $\text{floor}(\text{promedio del grupo } u + \text{promedio del grupo } v)$. Este proceso genera un grafo completo no dirigido con 20 nodos y 190 aristas, suficiente para aplicar MST con Kruskal.

3. Módulo A — Divide y Vencerás

Este módulo se encarga de: ordenar las solicitudes por tenure mediante MergeSort y responder consultas de búsqueda mediante búsqueda binaria.

Pseudocódigo de MergeSort:

```

MergeSort(A):
    si A tiene 0 o 1 elementos:
        retornar A
    dividir A en dos mitades: izquierda y derecha
    izquierda_ordenada = MergeSort(izquierda)
    derecha_ordenada = MergeSort(derecha)
    retornar Merge(izquierda_ordenada, derecha_ordenada)

Merge(L, R):
    crear arreglo resultado vacío
    mientras L y R no estén vacíos:
        tomar el menor primer elemento de L o R y moverlo a resultado
    agregar los elementos restantes de L o R
    retornar resultado

```

Pseudocódigo de búsqueda binaria:

```

SubProceso posicion <- BusquedaBinariaRekursiva(A, inicio, fin, k)
    Si inicio > fin Entonces
        posicion <- -1
    Sino
        medio <- trunc((inicio + fin) / 2)

        Si A[medio] = k Entonces
            posicion <- medio
        Sino
            Si A[medio] < k Entonces
                posicion <- BusquedaBinariaRekursiva(A, inicio, medio - 1, k)
            Sino
                posicion <- BusquedaBinariaRekursiva(A, medio + 1, fin, k)
        FinSi
    FinSi
FinSubProceso

```

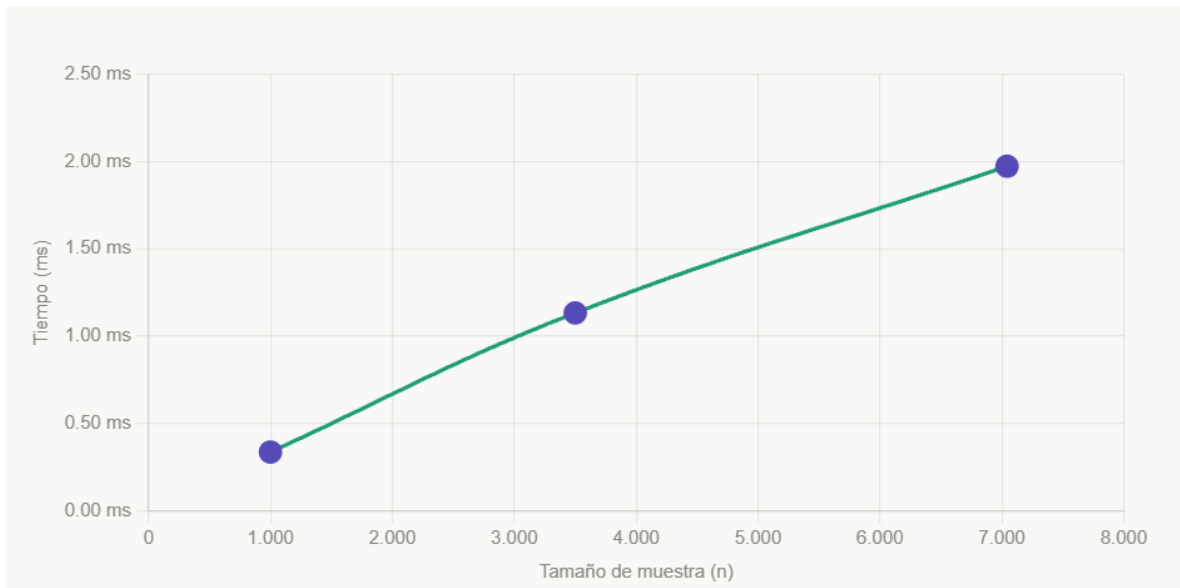
Análisis de Complejidad:

- MergeSort: $O(n \log n)$ en tiempo y $O(n)$ en espacio auxiliar.
- Búsqueda binaria: $O(\log n)$ en tiempo y $O(1)$ en espacio.

Tabla de tiempos empíricos para los tres tamaños de muestras

La tabla presenta los tiempos promedio medidos para MergeSort y búsqueda binaria sobre tres tamaños de muestra: pequeño, mediano y grande. Cada valor corresponde al promedio de 100 ejecuciones para reducir el ruido de la medición. Se observa que MergeSort crece de forma notable al aumentar el tamaño de la muestra, mientras que la búsqueda binaria mantiene un tiempo prácticamente constante por ser $O(\log n)$ y ejecutarse sobre un arreglo ya ordenado.

Tamaño de muestra	Tiempo MergeSort
1000	0.336451 ms
3500	1.13284 ms
7043	1.97321 ms



Como podemos observar en el gráfico, el tiempo crece según el tamaño de forma logarítmica acercándose a la forma $n \log n$

Resultados de las 5 consultas

k = 72 -> 5248-YGIJN (tenure = 72)

k = 60 -> 4667-QONEA (tenure = 60)

k = 45 -> 7795-CFOCW (tenure = 45)

k = 30 -> 6865-JZINKO (tenure = 30)

k = 12 -> 1680-VDCWW (tenure = 12)

4. Módulo B — Codicioso

Este módulo construye un MST con el algoritmo de Kruskal sobre el grafo generado a partir de los 20 grupos de solicitudes.

Pseudocódigo de Kruskal con Union-Find:

```
Kruskal(G):
    ordenar las aristas por peso ascendente
    crear una estructura Union-Find con un conjunto por vértice
    MST = vacío
    para cada arista (u, v) en orden:
        si find(u) != find(v):
            agregar arista al MST
            unir(u, v)
    retornar MST
```

El Union-Find permite consultar y actualizar componentes de manera eficiente, usando compresión de caminos y unión por rango o tamaño. La complejidad total del algoritmo es $O(E \log E)$ por el ordenamiento de aristas.

La demostración del lema del ciclo

En cualquier ciclo del grafo, la arista de mayor peso no pertenece a algún MST mínimo si existe una arista alternativa de menor o igual peso que conecte los mismos componentes. Kruskal siempre elige la arista más liviana que no forme ciclo.

Lista de aristas del MST y peso total

- Nodos: 20
- Aristas: 190
- Aristas del MST: 19
- Peso total del MST: 2393

```
ADA_PF_CORTES_ROLDAN > results > mst_red.txt
1  Minimum Spanning Tree (Kruskal)
2
3  11 - 17 : 124
4  13 - 17 : 124
5  2 - 17 : 125
6  17 - 19 : 125
7  7 - 17 : 125
8  14 - 17 : 125
9  0 - 17 : 125
10 15 - 17 : 126
11 3 - 17 : 126
12 10 - 17 : 126
13 1 - 17 : 126
14 6 - 17 : 126
15 9 - 17 : 126
16 8 - 17 : 127
17 12 - 17 : 127
18 16 - 17 : 127
19 17 - 18 : 127
20 5 - 17 : 127
21 4 - 17 : 129
22
23 Total Weight: 2393
```

5. Módulo C — Programación Dinámica

Este módulo resuelve el problema de la Mochila 0-1 para maximizar el ingreso mensual en un nodo con capacidad limitada de $W = 500$ unidades. Según los lineamientos técnicos, el peso (w_i) es el TotalCharges redondeado y el valor (v_i) es el MonthlyCharges multiplicado por 10.

Formulación de la recurrencia:

Sea $dp[i][w]$ el valor máximo posible usando los primeros i elementos con capacidad w .

$$dp[i][w] = \begin{cases} dp[i-1][w], & \text{si } peso_i > w \\ \max(dp[i-1][w], dp[i-1][w - peso_i] + valor_i), & \text{si } peso_i \leq w \end{cases}$$

Con condiciones iniciales:

$$dp[0][w] = 0, dp[i][0] = 0$$

Pseudocódigo de tabulación y backtracking:

```
Mochila0_1(solicitudes, W):
    n = 50
    Crear tabla dp[n + 1][W + 1] inicializada en 0

    // Fase 1: Tabulación
    Para i desde 1 hasta n:
        Para w desde 0 hasta W:
            Si pesos[i] <= w:
                // Decisión: elegir el máximo entre incluir o no la solicitud
                dp[i][w] = max(dp[i-1][w], valores[i] + dp[i-1][w - pesos[i]])
            Sino:
                dp[i][w] = dp[i-1][w]

    // El valor óptimo se encuentra en dp[n][W]
    Retornar dp[n][W]

ReconstruirSolucion(dp, pesos, n, W):
    w_actual = W
    Para i desde n hasta 1 (descendente):
        // Si el valor cambia, significa que el elemento i fue incluido
        Si dp[i][w_actual] != dp[i-1][w_actual]:
            Agregar solicitud[i] a la lista de seleccionados
            w_actual = w_actual - pesos[i]
```

Fallo del enfoque codicioso

Se demostró que elegir solicitudes basadas solo en la mejor relación valor/peso (v_i/w_i) no garantiza el óptimo global.

Tabla comparativa (Contraejemplo):

Artículo	Peso	Valor	Razón (V/P)
A	10	60	6.0
B	20	100	5.0
C	30	120	4.0

Enfoque codicioso

- Selecciona A (peso 10, valor 60) → Peso total: 10, Valor total: 60
- Selecciona B (peso 20, valor 100) → Peso total: 30, Valor total: 160
- No puede seleccionar C (peso 30, haría total 60 > 50)
- **Valor final codicioso: 160 centavos**

Solución óptima (programación dinámica):

- Selecciona B (peso 20, valor 100)
- Selecciona C (peso 30, valor 120)
- Peso total: 50, Valor total: 220
- **Valor final óptimo: 220 centavos**

La solución codicios obtiene 160 centavos, mientras que la solución óptima obtiene 220 centavos. Esto demuestra que tomar localmente la mejor opción no garantiza la mejor solución global.

Complejidad:

- Tiempo: $O(nW)$
- Espacio: $O(nW)$
- Es pseudopolinomial porque depende del valor numérico de W , no solo del tamaño del input en bits.

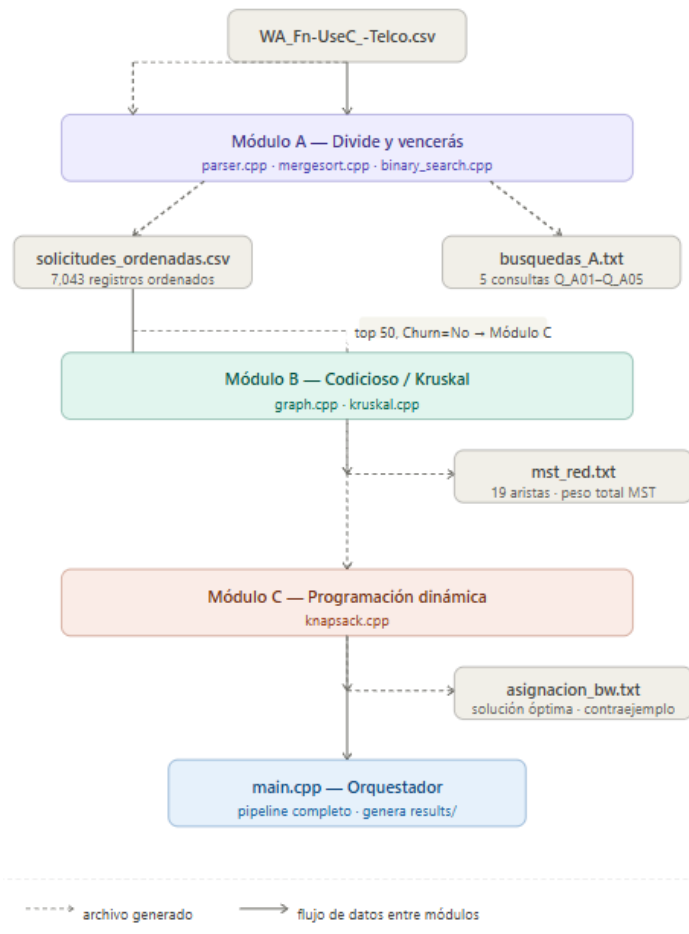
Discusión de pseudopolinomialidad

La complejidad temporal es $O(nW)$ y la complejidad espacial es $O(nW)$ si se conserva la tabla completa. Esta complejidad es pseudopolinomial porque depende del valor numérico de W y no solo del número de bits requeridos para representarlo. Por lo tanto, aunque el algoritmo es eficiente para instancias donde W es moderado, puede volverse computacionalmente costoso si la capacidad crece significativamente.

```
ADA_PF_CORTES_ROLDAN > results >  asignacion_bw.txt
54 Solucion optima (PD):
55 Valor total (en centavos): 6912
56 Peso total usado: 432
57 Numero solicitudes seleccionadas: 6
58 CustomerIDs seleccionados:
59 4376-KFVRS
60 1480-BKXGA
61 8606-CIQUL
62 1352-HNSAW
63 8207-DMRVL
64 6919-ELBGL
65
66 Seleccion voraz por ratio:
67 4376-KFVRS,1480-BKXGA,8606-CIQUL,1352-HNSAW,8207-DMRVL,6919-ELBGL,
68
69 Contraejemplo codicioso (3 items):
70 No se encontro contraejemplo de 3 items.
71
72 Analisis de complejidad:
73 Tiempo: Theta(n * W) porque se llena una tabla de n filas por W columnas.
74 Espacio: Theta(n * W) porque se almacena la tabla completa dp.
```

6. Integración del pipeline

Diagrama de flujo



7. Conclusiones:

El proyecto muestra distintos paradigmas resuelven problemas con propiedades diferentes. Divide y Vencerás funciona bien para ordenar y buscar porque divide el problema en subproblemas más simples y los combina eficientemente. El enfoque codicioso sí garantiza optimalidad en el MST porque el criterio de escoger aristas ligeras es seguro dentro del marco del lema del ciclo. En cambio, la mochila 0-1 requiere Programación Dinámica porque la decisión local no basta para garantizar la solución óptima global.

También se observa que los datos reales obligan a cuidar la escala de las variables. En este proyecto fue necesario ajustar la variable de peso para que la mochila fuera significativa. Esto demuestra que, más allá de la teoría, la representación de datos influye directamente en la calidad y utilidad del resultado.

8. Herramientas utilizadas

- Se utilizó la IA generativa para la consulta de conceptos teóricos sobre la recursividad en MergeSort, la lógica de la subestructura óptima en la Mochila 0-1 y la resolución de errores de

compilación en C++. No se realizó copia directa de código; toda la lógica fue rescrita y comprendida por el equipo para garantizar la capacidad de sustentación oral.

- las Diapositivas y material de apoyo de la clase de análisis y diseños de algoritmos
- Videos explicativos sobre la implementación de tabulación y backtracking en Programación Dinámica.

9. Referencias

- Dataset principal:

IBM Sample Data Sets. (2018). *Telco Customer Churn*. Kaggle. Recuperado de <https://www.kaggle.com/datasets/blastchar/telco-customer-churn>.

- G-Tech Education. (2021). *Kruskal's Algorithm in Data Structure*. [Archivo de Video]. Recuperado de: <https://youtu.be/IZHvQTx2bZ0>

- Programiz. (2020). *0/1 Knapsack Problem - Dynamic Programming*. [Archivo de Video]. Recuperado de: https://youtu.be/ip2jC_kXGtg

- Abdul Bari. (2018). *4.5.1 0/1 Knapsack Problem - Dynamic Programming*. [Archivo de Video]. Recuperado de: <https://youtu.be/e5zaZEfHsls>

- Anthropic. (2026). *Claude AI*. Recuperado de: <https://claude.ai/>

- Google. (2026). *Gemini AI*. Recuperado de: <https://gemini.google.com/>

- Microsoft. (2026). *Microsoft Copilot*. Recuperado de: <https://copilot.microsoft.com/>